LÖSUNGEN · LEHREREXEMPLAR

W3·2 – Dungeon-Karte, Flood-Fill und Backtracking: Lösungen

Zum Arbeitsblatt `AQ_W3_AB2_Dungeon-Karte_Flood-Fill_Backtracking.docx` · IDE: `AQ_W3_L3_Labyrinth_LSG.txt` (Klasse Karte) und `AQ_W3_DungeonRennen_LSG.txt` · alle Karten, Reihenfolgen, Aufrufzahlen und Endkarten mit Java (javac 21) nachgerechnet; findeWeg und lueften identisch mit der Musterlösung von Level 3.2 · Rekursionstiefe: Erstaufruf = 0

Teil 1 (DS 19)

Aufgabe 1 – Koordinaten im Kerker (6 BE)

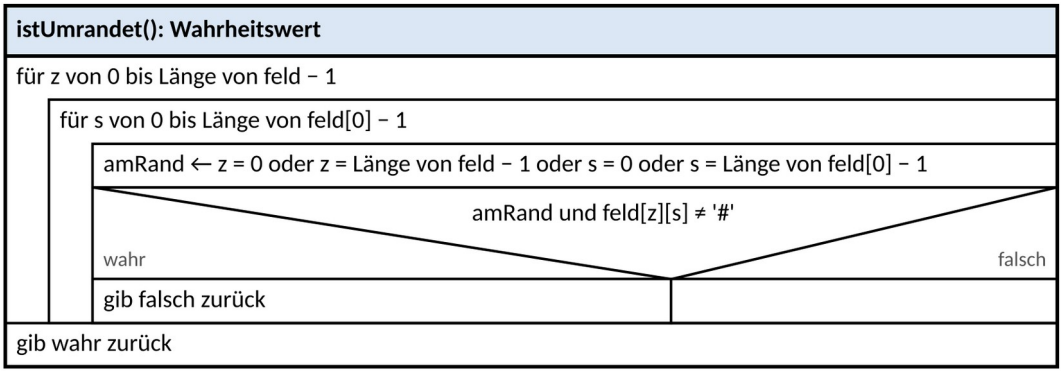
feld.length	feld[0].length	feld[2][5]	feld[4][3]	feld[3][4]	feld[1][6]
6	8	'Z'	'.'	'#'	'.'

b) Laufzeitfehler (ArrayIndexOutOfBoundsException) bei `feld[6][0]`, `feld[0][8]` und `feld[-1][2]`; `feld[5][7]` ist die Ecke unten rechts ('#'). **c)** (1|3) (2|3) (3|3) (4|3). **d)** (2|5) → feld[5][2] = '#' statt 'Z'; (4|3) → feld[3][4] = '#' statt '.'; (1|6) → feld[6][1]: **ArrayIndexOutOfBoundsException** (es gibt nur die Zeilen 0 bis 5). Auf quadratischen Karten liegen alle vertauschten Indizes noch im Feld – das Programm stürzt nicht ab, liefert aber die an der Diagonale gespiegelten Felder. Deshalb sind alle Testkarten des Kartenbuchs **nicht** quadratisch.

Aufgabe 2 – Rundgang durch die Karte (9 BE)

```
public int anzahlInSpalte(int s, char c) {
    int anzahl = 0;
    for (int z = 0; z < feld.length; z++) {      // nur die Zeilen laufen, s bleibt fest
        if (feld[z][s] == c) {
            anzahl++;
        }
    }
    return anzahl;
}
```

a) Kerker: anzahlInSpalte(3, '.') = **4** (weitere Kontrollwerte: anzahlInSpalte(1, '#') = 2, anzahlInSpalte(6, '.') = 3). Häufige Fehler: zweite Schleife über die Spalten, `z <= feld.length`, Vergleich mit equals statt == (3 BE).



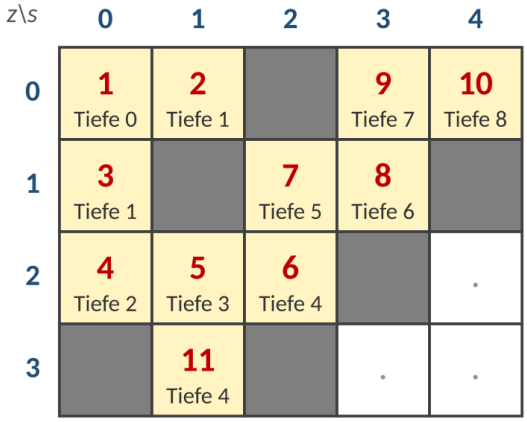
b) Kerker → **wahr**, Lichtung → **falsch** (mit Java geprüft: Kerker true, Lichtung false, Höhle true, Labyrinth true). Gleichwertig: vier einzelne Schleifen über die Randzeilen und -spalten; wichtig sind die Grenzen Länge − 1 und die Rückgabe wahr erst **nach** den Schleifen (4 BE). **c)** Die Schleifen laufen spaltenweise: Jede Ausgabezeile ist eine Spalte der Karte (die Karte wird an der Diagonale gespiegelt): `..#` · `.S.` · `#..` · `..Z` (2 BE).

Aufgabe 3 – Wer ist mein Nachbar? (6 BE)

Feld	oben	rechts	unten	links	begehbar
(0|1)	(-1|1) –	(0|2) '#'	(1|1) 'S'	(0|0) '.'	2
(2|0)	(1|0) '.'	(2|1) '.'	(3|0) –	(2|-1) –	2
(1|3)	(0|3) '.'	(1|4) –	(2|3) 'Z'	(1|2) '.'	3

b) Kerker: (1|2) → **2**, (3|1) → **3**, (4|6) → **1**, (2|5) → **2** ('S' und 'Z' zählen als begehbar). **c)** Ecke **3**, Rand **5**, innen **8**.

Aufgabe 4 – Nebel lüften in der Wachstube (12 BE)



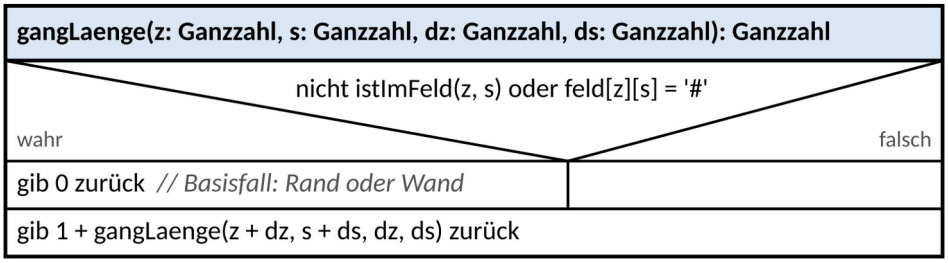
a) Reihenfolge und Tiefe siehe Abbildung (4 BE): 1: (0|0) T0 · 2: (0|1) T1 · 3: (1|0) T1 · 4: (2|0) T2 · 5: (2|1) T3 · 6: (2|2) T4 · 7: (1|2) T5 · 8: (1|3) T6 · 9: (0|3) T7 · 10: (0|4) T8 · 11: (3|1) T4.

b) Im Nebel bleiben (2|4), (3|3) und (3|4): Sie sind nur über die Wände (1|4), (2|3) und (3|2) bzw. diagonal mit dem Rest verbunden – kein 4er-Nachbar ist gelüftet (1 BE).

c) m = 11 markierte Felder → **45 Aufrufe** = 1 + 4 · 11. Begründung: Nur ein Aufruf, der sein Feld markiert, ruft weitere Aufrufe auf – genau vier. Jedes Feld wird höchstens einmal markiert. Alle Aufrufe außer dem Erstaufruf stammen also von markierenden Aufrufen: 1 + 4 · m. Die Anzahl der Aufrufe wächst linear mit der Zahl der erreichbaren Felder (3 BE). Größte Rekursionstiefe beim Markieren: **8**, bei den sofort endenden Aufrufen 9.

d) fackel(0, 0) markiert (0|0) nicht sofort und beauftragt zuerst oben (außerhalb), dann rechts (0|1). (0|1) ist noch '.', markiert ebenfalls nicht, beauftragt oben (außerhalb), rechts und unten (Wände) und links **(0|0)** – das noch '.' ist und wieder (0|1) aufruft. Die beiden Felder rufen sich endlos gegenseitig auf → StackOverflowError; es wird nie ein Feld markiert (2 BE). **e)** Das **Ergebnis** bleibt gleich: Gelüftet wird genau die Menge der von (0|0) aus erreichbaren Felder, egal in welcher Reihenfolge. Die **Nummerierung** und die Tiefen ändern sich (z. B. mit „links, unten, rechts, oben“ käme (1|0) vor (0|1)) (2 BE).

Aufgabe 5 – Abi-Format: gangLaenge (9 BE)



a) Kernpunkte (4 BE): Kopf mit Signatur · Basisfall „außerhalb oder Wand → 0“ **vor** dem Feldzugriff (Kurzschluss: „oder“ wertet den Zugriff nur aus, wenn (z|s) im Feld liegt) · Selbstaufruf mit (z + dz | s + ds) und unveränderter Richtung · Rückgabe 1 + Ergebnis des Selbstaufrufs. Gleichwertig: Verzweigung mit „gib 0 zurück“ / „gib 1 + … zurück“ in zwei Zweigen.

Rekursionstiefe	Aufruf	Zeichen	Rückgabe
0	gangLaenge(1, 1, 1, 0)	'S'	4
1	gangLaenge(2, 1, 1, 0)	'.'	3

Rekursionstiefe	Aufruf	Zeichen	Rückgabe
2	gangLaenge(3, 1, 1, 0)	'.'	2
3	gangLaenge(4, 1, 1, 0)	'.'	1
4	gangLaenge(5, 1, 1, 0)	'#'	0 (Basisfall)

b) Rückgabe **4** (3 BE). Kontrollwerte: gangLaenge(4, 3, 0, 1) = 4, gangLaenge(1, 1, 0, 1) = 3, gangLaenge(0, 2, 1, 0) = 0 (Wand), Lichtung gangLaenge(1, 0, 0, 1) = 4 (bis zum Rand). **c)** Jeder Selbstaufruf geht genau ein Feld in dieselbe Richtung (dz|ds) ≠ (0|0) weiter. Spätestens nach so vielen Schritten, wie die Karte Zeilen bzw. Spalten hat, liegt das Feld außerhalb – dann greift der Basisfall. Ohne Randmauer ist genau deshalb die Prüfung istImFeld nötig; eine Besucht-Markierung braucht man nicht, weil nie zurückgegangen wird (2 BE).

Aufgabe 6 – Level 3.2

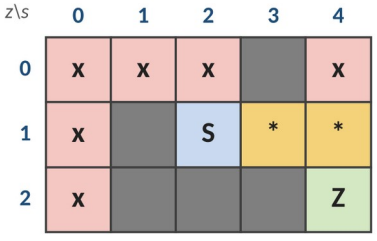
Musterlösungen und häufige Fehler: `AQ_W3_L3_Labyrinth_LSG.txt`, Klasse Karte (lueften: vorübergehend '+', danach S/Z zurücksetzen). Nebel lüften im Rennen-Gerüst (Reihenfolge oben, rechts, unten, links; mit einem Java-Nachbau auf den Rennkarten nachgerechnet, identisch mit `AQ_W3_DungeonRennen_LSG.txt`):

Rennkarte	gelüftet m	Prüfungen	1 + 4 · m	korrekt?
1 (klein)	30	121	121	ja
2 (mittel)	65	261	261	ja (Kammer bleibt grau)
3 (groß)	140	561	561	ja

Teil 2 (DS 20, Kür)

Aufgabe 7 – Der Ariadnefaden im Gewölbe (11 BE)

Rekursionstiefe	Richtung	Weg
0	– (Aufruf)	(1|2)
1	oben	(1|2)(0|2)
2	links	(1|2)(0|2)(0|1)
3	links	(1|2)(0|2)(0|1)(0|0)
4	unten	(1|2)(0|2)(0|1)(0|0)(1|0)
5	unten	(1|2)(0|2)(0|1)(0|0)(1|0)(2|0)
1	rechts	(1|2)(1|3)
2	rechts	(1|2)(1|3)(1|4)
3	oben	(1|2)(1|3)(1|4)(0|4)
3	unten	(1|2)(1|3)(1|4)(2|4) (Ziel erreicht)



a) 10 Zeilen (6 BE; je fehlerhafte Zeile −1). Die lange Sackgasse oben links wird zuerst vollständig erkundet (Tiefe bis 5), erst danach geht es von S nach rechts. Die Geschwister „oben“ und „unten“ in Tiefe 3 haben dieselbe Tiefe. Insgesamt 32 Aufrufe von findeWeg (schritte), auch auf Wänden und außerhalb.

b) Endkarte siehe Abbildung: Weg S → (1|3) → (1|4) → Z mit `'*'`, Sackgassen (0|0), (0|1), (0|2), (1|0), (2|0) und (0|4) mit 'x' (2 BE).

c) Ein Feld wird beim Betreten vorläufig `'*'`. Führt von dort kein Nachbar zum Ziel, setzt `feld[z][s] ← 'x'` es auf Sackgasse: Der Schritt wird zurückgenommen, das Feld gehört nicht mehr zum Weg und bleibt als besucht markiert (wird nicht erneut betreten). Ohne die Anweisung blieben (0|2), (0|1), (0|0), (1|0), (2|

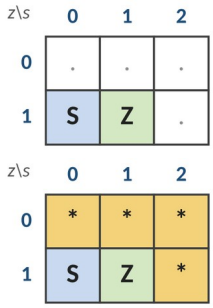
0) und (0|4) `'*'` – nach dem Ende stünde ein zerrissener, falscher „Weg“ auf der Karte, obwohl findeWeg korrekt wahr liefert (3 BE).

Aufgabe 8 – Sackgassen auf der Lichtung (8 BE)

a) Von S aus wird zuerst **oben** versucht: (0|1) betreten. Von dort ist oben außerhalb, rechts Wand, unten S (schon `'*'`), links (0|0) wird betreten; von dort unten (1|0) betreten. Von (1|0) aus ist kein Nachbar mehr frei → falsch. Da kein Nachbar zum Ziel führt, nimmt jeder dieser Aufrufe seinen Schritt zurück: betreten in der Reihenfolge (0|1), (0|0), (1|0), zurückgenommen umgekehrt (1|0), (0|0), (0|1) – wie beim Stapel (3 BE).

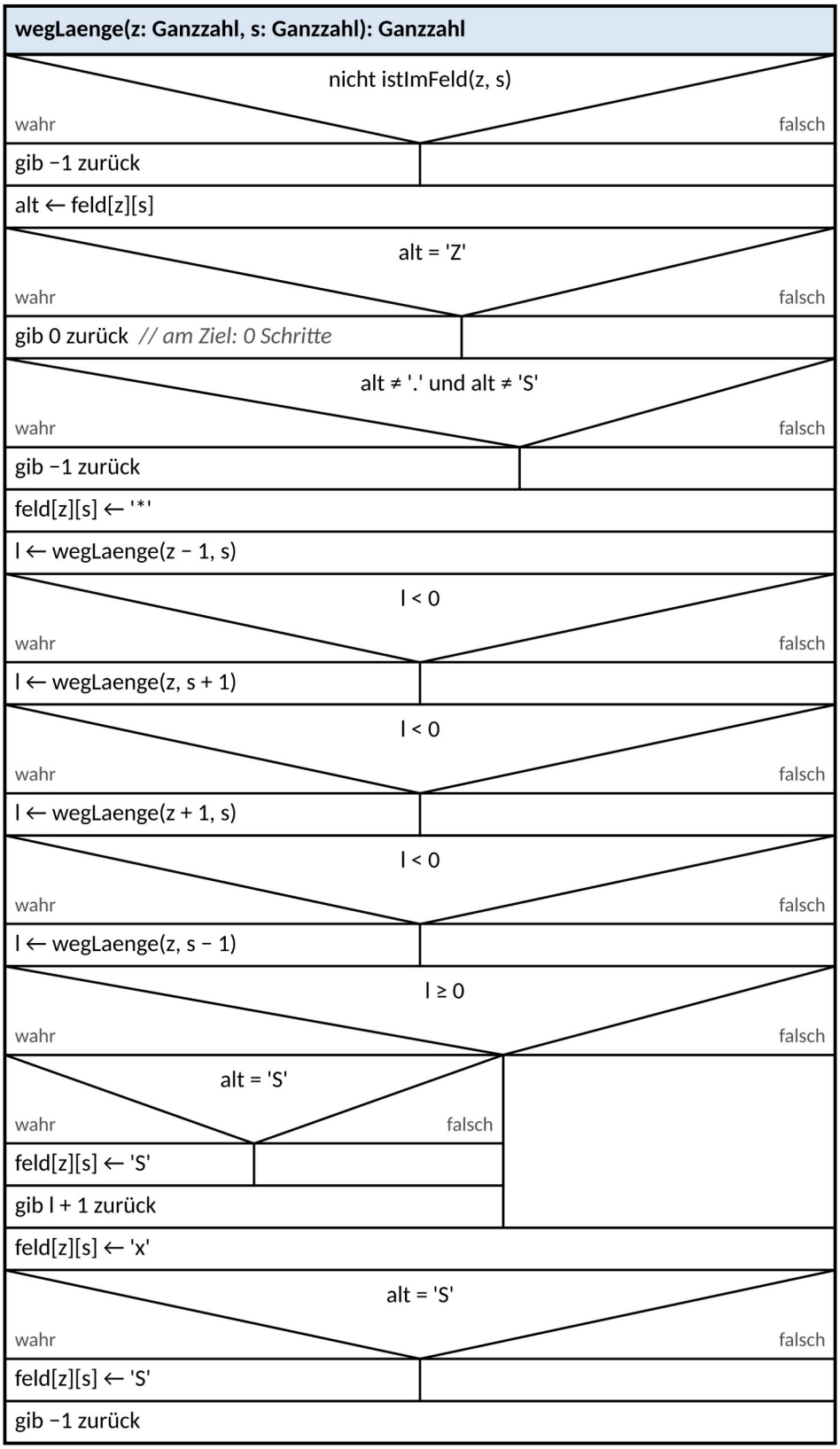
b) Danach probiert S **rechts**: (1|2) → (1|3) → oben (0|3) ist Sackgasse → unten Z: wahr. Weil „oder“ beim ersten wahr abbricht, werden von S aus „unten“ (2|1) und „links“ nie mehr versucht; (2|1) und (2|2) werden nie betreten und bleiben '.' (1 BE). Vollständige Tabelle der Lichtung: Warm-up W3-08 (Sicherung DS 20), 24 Aufrufe.

c) findeWeg endet trotzdem immer: Die Felder des aktuellen Weges sind `'*'` und werden nicht erneut betreten, es gibt also keine Kreise, und es gibt nur endlich viele kreisfreie Wege. Aber Sackgassen sind auf der Karte nicht mehr zu erkennen und können über einen anderen Weg erneut erkundet werden – die Zahl der Aufrufe kann stark steigen (Java: Lichtung, Höhle, Labyrinth unverändert 24/15/41, **Kerker 101 statt 53** Aufrufe, weil der Ring um die Wand (2|2) doppelt durchlaufen wird) (2 BE).



d) Gegenbeispiel (2 × 3): S auf (1|0), Z direkt rechts daneben. findeWeg probiert zuerst **oben** und läuft über (0|0), (0|1), (0|2), (1|2) zum Ziel: **5 Schritte** statt 1 (Endkarte rechts, 13 Aufrufe). Backtracking findet **irgendeinen** Weg; welcher es ist, hängt von der Reihenfolge der Richtungen ab (2 BE).

Aufgabe 9 – Abi-Format: wegLaenge (8 BE)



a) Kernpunkte (5 BE): Rückgabetyp Ganzzahl · Basisfälle −1 (außerhalb, Wand/besucht) und **0 am Ziel** · Markierung `'*'` vor den Selbstaufrufen · Nachbarn nacheinander nur, solange noch kein Weg gefunden ist (l < 0) – entspricht dem Abbruch von „oder“ · bei Erfolg **l + 1** liefern (ein Schritt mehr) und S wiederherstellen · sonst 'x' und −1. Gleichwertig: vier Verzweigungen „falls l ≥ 0: gib l + 1 zurück“ hintereinander. Fehler: schritte mitgezählt (kein Fehler, aber überflüssig), Rückgabe l statt l + 1, Ziel liefert 1.

b) Kerker: **-1** (kein Weg), Gewölbe: **3** (mit Java geprüft; ebenso Lichtung 3, Höhle 6, Labyrinth 14) (1 BE). **c)** wegLaenge liefert die Länge des **ersten gefundenen** Weges; die Suche bricht beim ersten Erfolg ab und vergleicht keine Alternativen. Auf der Karte aus Aufgabe 8 d) liefert wegLaenge(1, 0) den Wert 5, der kürzeste Weg hat 1 Schritt. (Auf Lichtung, Höhle und Labyrinth stimmen die Werte zufällig mit den kürzesten Wegen 3, 6, 14 überein.) (2 BE)

Aufgabe 10 – Level 3.2 Experte

Musterlösung, Lichtung-Trace (24 Aufrufe) und häufige Fehler: `AQ_W3_L3_Labyrinth_LSG.txt`. Kontrollwerte: Höhle 15, Labyrinth 41, Kerker 53, Lichtung 24 Aufrufe.

Aufgabe 11 – Das Dungeon-Rennen

Referenzwerte (Summe der Schritte auf den drei Rennkarten, aus `AQ_W3_DungeonRennen_LSG.txt`, dort mit `Referenz.tabelle()` gemessen – vor dem Einsatz einmal nachmessen): Standard (oben, rechts, unten, links) **160** (23 / 49 / 88) · beste feste Reihenfolge **152** · Kompass „Richtung Ziel zuerst“ **139** (13 / 53 / 73) · kürzester Weg bei offener Karte 83 (nur untere Schranke). **c)** Z. B. Kompass auf Karte 2: Das Ziel liegt unten links, genau dort beginnt eine lange Sackgasse – 53 statt 49 Schritte. Eine Heuristik ist eine Faustregel, keine Garantie. Ergebnisse an der Tafel festhalten (Team, Schritte, gültig ja/nein) – Team-Wettbewerb ohne Lab-Wertung; das beste gültige Team stellt seine Strategie vor.

Für Schnelle

8er-Nachbarschaft: Im Labyrinth werden zusätzlich (6|6) (7|5) (7|6) (7|7) gelüftet (29 statt 25 Felder '+'): Die Kammer hängt über Eck an (5|5) und am Ziel. **Tiefe:** Auf einer leeren n × n-Karte läuft fackel(0, 0) als Schlange durch die ganze Karte; die größte Tiefe ist $n^2 - 1$ (gemessen n = 2 … 6: 2 → 3, 3 → 8, 4 → 15, 5 → 24, 6 → 35), für 4 × 4 also **15**. Bei sehr großen Karten droht ein Stapelüberlauf. **Kompass:** Rennkarte 2 (siehe oben).